

將以 Jib 容器化的 Micronaut 應用程式部署至 Google Kubernetes Engine

來源：<https://codelabs.developers.google.com/codelabs/cloud-micronaut-kubernetes?hl=zh-tw>

步驟數：15

在本程式碼研究室中，您將瞭解如何將 Microaut 微服務轉換為在 Google Kubernetes Engine 上執行的複製服務。

目錄

1. [1. 總覽](#)
2. [2. 設定和需求](#)
3. [3. 取得 Micronaut 範例原始碼](#)
4. [4. 快速瀏覽程式碼](#)
5. [5. 在本機執行應用程式](#)
6. [6. 使用 Jib 將應用程式封裝為 Docker 容器](#)
7. [7. 將容器化服務推送至登錄檔](#)
8. [8. 建立叢集](#)
9. [9. 將應用程式部署至 Kubernetes](#)
10. [10. 允許外部流量](#)
11. [11. 擴充服務](#)
12. [12. 推出服務升級](#)
13. [13. 復原](#)
14. [14. 摘要](#)
15. [15. 恭喜！](#)

1. 總覽

關於 Micronaut

Micronaut 是以 JVM 為基礎的現代化全端架構，可建構模組化且易於測試的微服務和無伺服器應用程式。Micronaut 的目標是提供優異的啟動時間、快速的總處理量，以及最小的記憶體用量。開發人員可以使用 Java、**Groovy** 或 Kotlin 透過 Micronaut 進行開發。

Micronaut 提供：

- **啟動時間快速且記憶體用量低**：以反射為基礎的 IoC 架構會載入並快取程式碼中每個欄位、方法和建構函式的反射資料，但使用 Micronaut 時，應用程式啟動時間和記憶體用量不會受程式碼大小限制。
- **宣告式、反應式、編譯階段 HTTP 用戶端**：宣告式建構反應式 HTTP 用戶端，這些用戶端會在編譯階段實作，減少記憶體耗用量。
- **以 Netty 為基礎建構的非封鎖 HTTP 伺服器**：Micronaut 的 HTTP 伺服器學習曲線平緩，可盡可能輕鬆地公開 API，供 HTTP 用戶端使用。
- **快速輕鬆地進行測試**：在單元測試中輕鬆啟動伺服器和用戶端，並立即執行。
- **有效率的編譯時間依附元件插入和 AOP** - Micronaut 提供簡單的編譯時間面向導向程式設計 API，不會使用反射。
- **建構完全回應式且非封鎖的應用程式**：Micronaut 支援任何實作 Reactive Streams 的架構，包括 RxJava 和 Reactor。

詳情請參閱 [Micronaut 網站](#)。

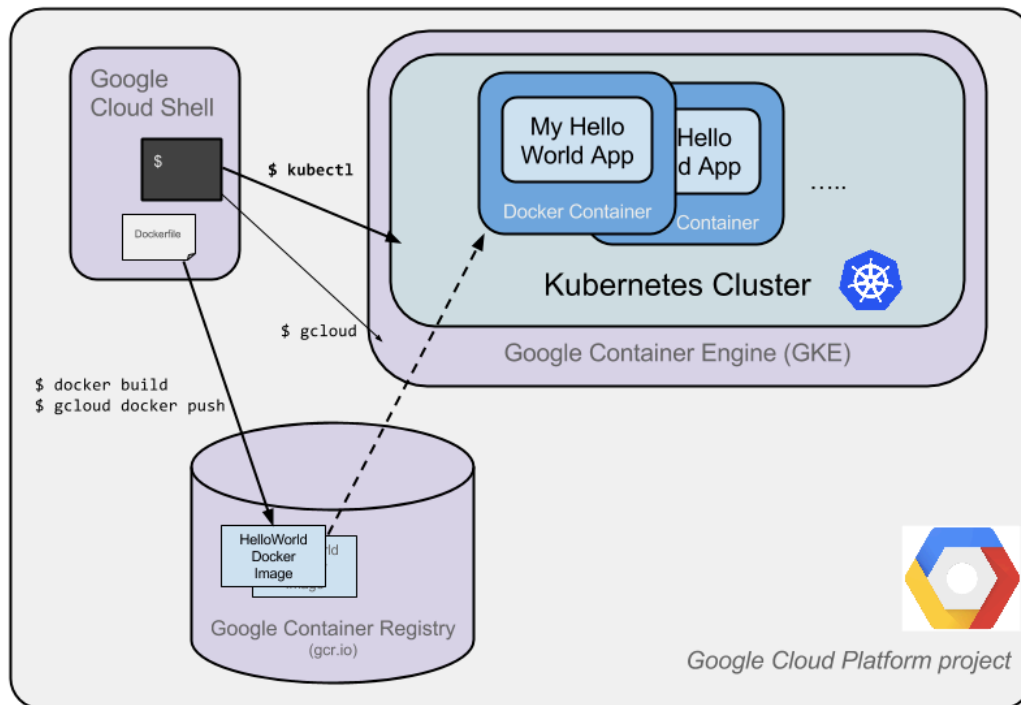
關於 Kubernetes

Kubernetes 是開放原始碼專案，能夠在許多不同環境中運作，包括筆記型電腦、高可用性的多節點叢集、公有雲、地端部署、虛擬機器和裸機環境。

在本實驗室中，您會將以 Groovy 為基礎的簡易 Micronaut 微服務，部署至在 **Kubernetes Engine** 上執行的 **Kubernetes**。

本程式碼研究室的目標是讓您以在 Kubernetes 上執行的複製服務形式，執行微服務。您會將在本機開發的程式碼轉換為 Docker 容器映像檔，然後在 Kubernetes Engine 上執行該映像檔。

下圖說明本程式碼研究室中各個部分的運作方式，協助您瞭解各個部分如何搭配運作。在完成本程式碼研究室的過程中，請將這份檔案做為參考資料；您應該會在完成時瞭解所有內容 (但現在可以忽略這份檔案)。



在本程式碼研究室中，使用 Kubernetes Engine (在 Compute Engine 上執行的 Google 代管 Kubernetes 版本) 等代管環境，可讓您專心體驗 Kubernetes，不必費心設定基礎架構。

如果您想在本機 (例如開發筆電) 上執行 Kubernetes，建議您瞭解 [Minikube](#)。這項服務可輕鬆設定單一節點 Kubernetes 叢集，用於開發和測試。如要使用 Minikube 完成本程式碼研究室，請參閱這篇文章。

關於 Jib

Jib 是一項開放原始碼工具，可讓您為 Java 應用程式建構 Docker 和 OCI 映像檔。這項工具提供 Maven 和 Gradle 外掛程式，以及 Java 程式庫。

Jib 的目標是：

- **快速：**快速部署變更。Jib 會將應用程式分成多個層，並將依附元件與類別分開。現在您不必等待 Docker 重建整個 Java 應用程式，只要部署變更的層即可。
- **可重現：**使用相同內容重建容器映像檔時，一律會產生相同映像檔。從此不必再觸發不必要的更新。
- **無精靈：**減少 CLI 依附元件。從 Maven 或 Gradle 內建構 Docker 映像檔，並推送到您選擇的任何登錄檔。您不必再編寫 Dockerfile，也不必呼叫 `docker build/push`。

如要進一步瞭解 Jib，請前往 Github [專案頁面](#)。

關於本教學課程

本教學課程使用 [Jib](#) 工具的範例程式碼，為 Java 應用程式建構容器。

這個範例是簡單的「hello world」服務，使用 [Micronaut](#) 架構和 [Apache Groovy](#) 程式設計語言。

課程內容

- 如何使用 Jib 將簡單的 Java 應用程式封裝為 Docker 容器
- 瞭解如何在 Kubernetes Engine 上建立 Kubernetes 叢集。
- 如何將 Micronaut 服務部署至 Kubernetes Engine 上的 Kubernetes
- 如何擴充服務及推出升級版本。
- 如何存取 Kubernetes 圖形資訊主頁。

軟硬體需求

- Google Cloud Platform 專案
 - [Chrome](#) 或 [Firefox](#) 瀏覽器
 - 熟悉標準 Linux 文字編輯器，例如 Vim、EMAC 或 Nano
-

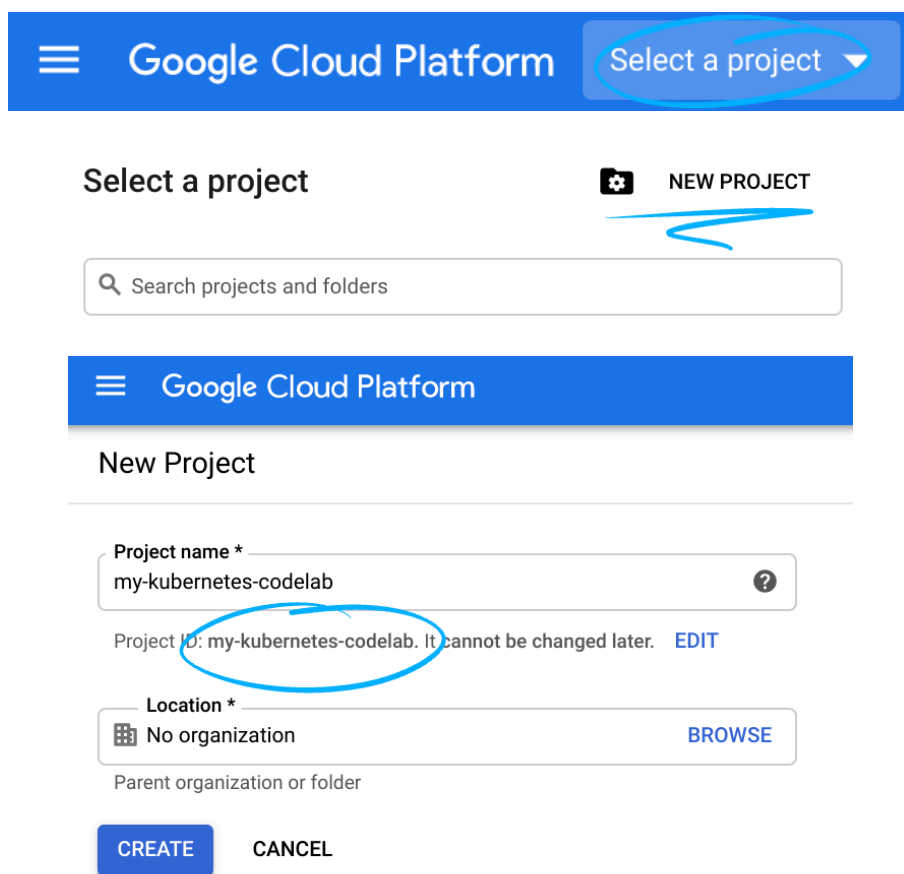
步驟 2 · 約 1 分鐘

2. 設定和需求

自修實驗室環境設定

1. 登入 [Cloud 控制台](#)，建立新專案或重複使用現有專案。(如果沒有 Gmail 或 G Suite 帳戶，請先[建立帳戶](#))。

注意：記住以下網址，即可輕鬆存取 Cloud 控制台：console.cloud.google.com。



The screenshot shows the Google Cloud Platform interface for creating a new project. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' dropdown. Below this, the 'Select a project' section features a search bar and a 'NEW PROJECT' button. The 'New Project' form is displayed below, with fields for 'Project name' (containing 'my-kubernetes-codelab'), 'Project ID' (containing 'my-kubernetes-codelab', which is circled in blue and has a note that it cannot be changed later), and 'Location' (set to 'No organization'). At the bottom of the form are 'CREATE' and 'CANCEL' buttons.

請記住專案 ID，這是所有 Google Cloud 專案中不重複的名稱 (上述名稱已遭占用，因此不適用於您，抱歉！)。本程式碼研究室稍後會將其稱為 `PROJECT_ID`。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 G Suite 帳戶，請選擇適合貴機構的位置。

2. 接著，您必須在 Cloud 控制台中[啟用帳單](#)，才能使用 Google Cloud 資源。

完成本程式碼研究室的費用應該不高，甚至完全免費。請務必按照「清除」部分的指示操作，瞭解如何停用資源，避免在本教學課程結束後繼續產生帳單費用。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

步驟 3 · 約 2 分鐘

3. 取得 Micronaut 範例原始碼

啟動 Cloud Shell 後，您可以使用指令列複製主目錄中的範例原始碼，然後 cd 到包含範例服務的目錄：

```
$ git clone https://github.com/GoogleContainerTools/jib.git  
$ cd jib/examples/micronaut/
```

[Jib 專案存放區](#)也包含其他架構的範例專案，例如 Spring Boot 或 Vert.x，方便您試用其他範例。

步驟 4 · 約 2 分鐘

4. 快速瀏覽程式碼

我們的 Micronaut 簡易服務是由控制器組成，可輸出著名的 Hello World 訊息：

```
@Controller("/hello")
class HelloController {
    @Get("/")
    String index() {
        "Hello World"
    }
}
```

`HelloController` 控制器會回應 `/hello` 路徑下的要求，而 `index()` 方法會接受 HTTP GET 要求。

您也可以使用 [Spock](#) 測試類別，確認輸出內容是否包含正確訊息。

```
class HelloControllerSpec extends Specification {
    @Shared
    @AutoCleanup
    EmbeddedServer embeddedServer = ApplicationContext.run(EmbeddedServer)

    @Shared
    @AutoCleanup
    RxHttpClient client = embeddedServer.applicationContext.createBean(RxHttpClient,
        embeddedServer.getURL())

    void "test hello world response"() {
        when:
        HttpRequest request = HttpRequest.GET('/hello')
        String rsp = client.toBlocking().retrieve(request)

        then:
        rsp == "Hello World"
    }
}
```

這項測試不僅僅是簡單的單元測試，實際上還會執行與正式環境相同的 Micronaut 伺服器堆疊 (以 [Netty](#) 架構為基礎)。因此，程式碼在產品中的行為與測試時完全相同。

如要執行測試，可以執行下列指令，確認一切正常：

```
./gradlew test
```


步驟 5 · 約 2 分鐘

5. 在本機執行應用程式

您可以使用下列 Gradle 指令，正常啟動 Micronaut 服務：

```
$ ./gradlew run
```

應用程式啟動後，您可以透過「+」小圖示開啟額外的 Cloud Shell 執行個體，然後使用 curl 檢查是否取得預期輸出內容：

```
$ curl localhost:8080/hello
```

您應該會看到簡單的「Hello World」訊息。

步驟 6 · 約 4 分鐘

6. 使用 Jib 將應用程式封裝為 Docker 容器

接著，準備在 Kubernetes 上執行應用程式。為此，我們將利用 Jib 為我們完成繁重的工作，因為我們不必自行處理 `Dockerfile` ！

執行指令來建構容器：

```
$ ./gradlew jibDockerBuild
```

您應該會看到下列輸出內容：

```
Tagging image with generated image reference micronaut-jib:0.1. If you'd like to
specify a different tag, you can set the jib.to.image parameter in your build.gradle,
or use the --image=<MY IMAGE> commandline flag.

Containerizing application to Docker daemon as micronaut-jib:0.1...
warning: Base image 'gcr.io/distroless/java' does not use a specific image digest -
build may not be reproducible
Getting base image gcr.io/distroless/java...
Building dependencies layer...
Building resources layer...
Building classes layer...
Finalizing...

Container entrypoint set to [java, -cp, /app/resources:/app/classes:/app/libs/*,
example.micronaut.Application]
Loading to Docker daemon...

Built image to Docker daemon as micronaut-jib:0.1
```

Jib 會建構具有不同層的 Docker 映像檔：依附元件、資源和已編譯的程式碼。舉例來說，如果您修改程式碼，系統只會修改含有已編譯類別的層，並重複使用未變更的先前層，藉此節省建構時間。

映像檔建構完成後，請在 Cloud Shell 的第一個分頁中執行 Docker 映像檔，確認是否能看到友善的問候訊息：

```
$ docker run -it -p 8080:8080 micronaut-jib:0.1
16:57:20.255 [main] INFO i.m.context.env.DefaultEnvironment - Established active
environments: [cloud, gcp]
16:57:23.203 [main] INFO io.micronaut.runtime.Micronaut - Startup completed in
2926ms. Server Running: http://97b7d76ccf3f:8080
```

服務正在執行中，因此我們現在可以在第二個 Cloud Shell 分頁中啟動 `curl` 指令，確認服務是否正常運作：

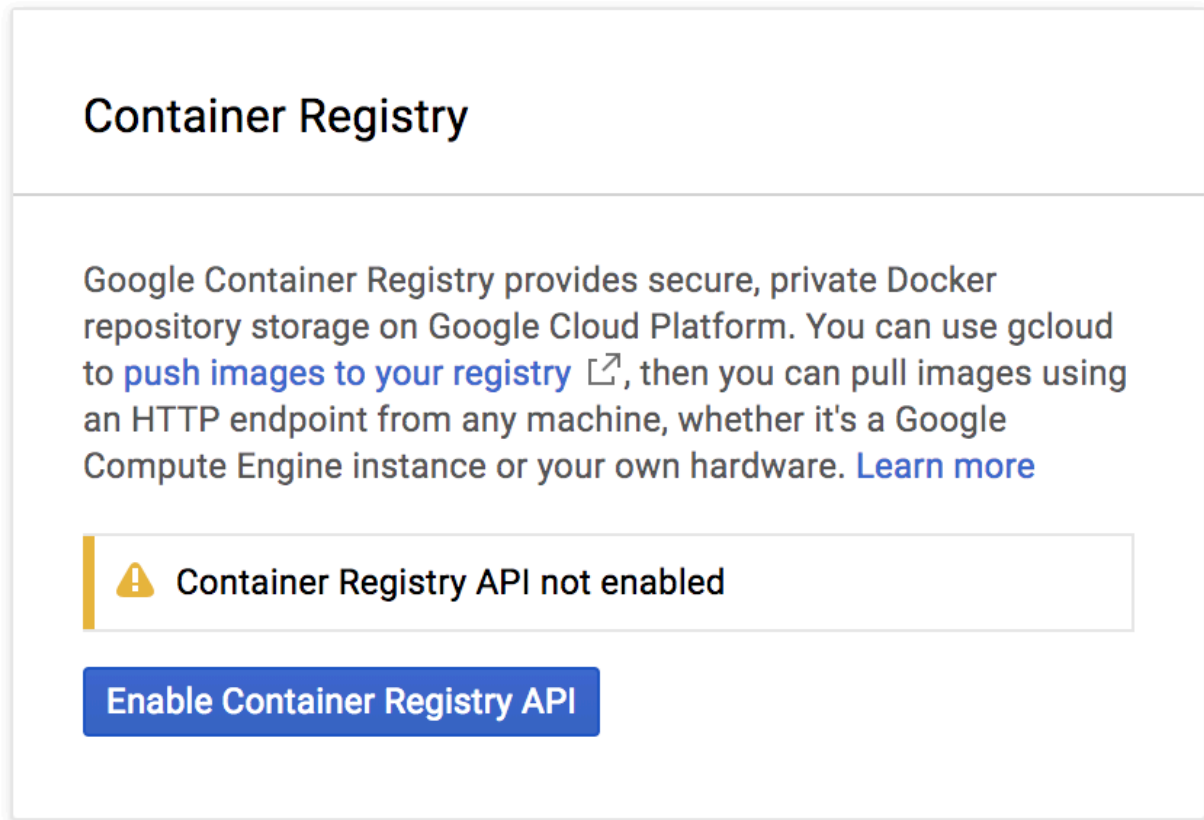
```
$ curl localhost:8080/hello  
Hello World
```

您可以在 Cloud Shell 中按下 `ctrl+c` 停止容器。

7. 將容器化服務推送至登錄檔

映像檔現在可正常運作，您可以將其推送至 [Google Container Registry](#)，這是 Docker 映像檔的私人存放區，可從每個 Google Cloud 專案存取 (但也可從 Google Cloud Platform 外部存取)。

在推送至登錄檔之前，請先前往「Tools」>「Container Registry」，確保專案已啟用 Container Registry。如果尚未啟用，您應該會看到下列對話方塊，請點選「啟用 Container Registry API」來啟用：



登錄檔準備就緒後，請啟動下列指令，將映像檔推送至登錄檔：

```
$ gcloud auth configure-docker
$ docker tag micronaut-jib:0.1 \
    gcr.io/$GOOGLE_CLOUD_PROJECT/micronaut-jib:0.1
$ docker push gcr.io/$GOOGLE_CLOUD_PROJECT/micronaut-jib:0.1
```

上述指令可讓 gcloud SDK 設定及授權 Docker，將映像檔推送至 Container Registry 執行個體，為映像檔加上標記以指向登錄檔中的位置，然後將映像檔推送至登錄檔。

如果一切順利，過一會兒後，您應該就能在控制台中看到列出的容器映像檔：依序點選「工具」>「Container Registry」。此時，您應該會看到專案適用的 Docker 映像檔，Kubernetes 幾分鐘後就能存取及協調這個映像檔。

請注意，雖然我們在此使用了登錄檔的通用網域 (`gcr.io`)，但您也可以更明確地指定要使用的可用區和值區。詳情請參閱：https://cloud.google.com/container-registry/docs/#pushing_to_the_registry



micronaut-jib

gcr.io / mn-gke-test / micronaut-jib



Filter by name or tag



Columns ▼



Name

Tags

Uploaded ▼



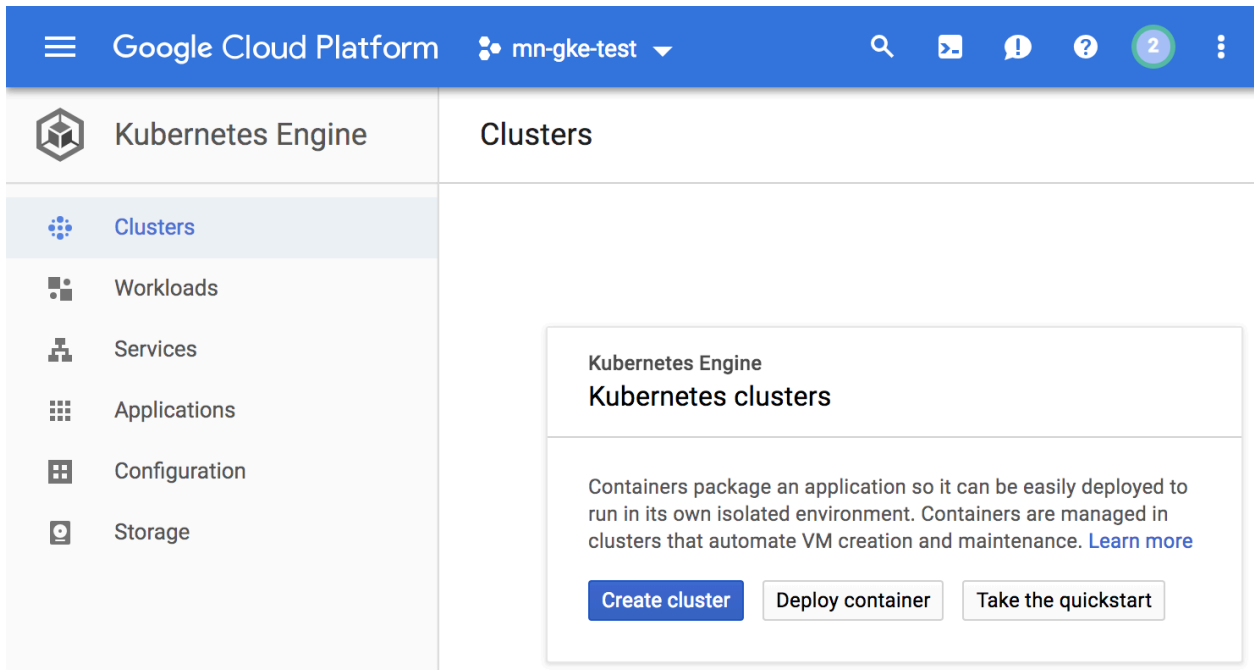
9fcb619a22d1

0.1

3 minutes ago

8. 建立叢集

現在可以建立 Kubernetes Engine 叢集了，但在此之前，請先前往網路控制台的 Google Kubernetes Engine 專區，等待系統初始化 (應該只需要幾秒鐘)。



叢集由 Google 管理的 Kubernetes 主要 API 伺服器和一組工作站節點組成。工作站節點是 Compute Engine 虛擬機器。讓我們使用 Cloud Shell 工作階段的 `gcloud` CLI，建立含兩個 `n1-standard-1` 節點的叢集 (這項作業需要幾分鐘才能完成)：

```
$ gcloud container clusters create hello-cluster \
  --num-nodes 2 \
  --machine-type n1-standard-1 \
  --zone us-central1-c
```

最後，您應該會看到建立的叢集。

```
Creating cluster hello-cluster in us-central1-c...done.
Created [https://container.googleapis.com/v1/projects/mn-gke-test/zones/us-central1-c/clusters/hello-cluster].
To inspect the contents of your cluster, go to:
https://console.cloud.google.com/kubernetes/workload_/gcloud/us-central1-c/hello-cluster?project=mn-gke-test
kubeconfig entry generated for hello-cluster.
```

NAME	LOCATION	MASTER_VERSION	MASTER_IP	MACHINE_TYPE
hello-cluster	us-central1-c	1.9.7-gke.7	35.239.224.115	n1-standard-1
1.9.7-gke.7	2	RUNNING		

或者，您也可以透過上述的控制台建立這個叢集：依序點選「Compute」>「Kubernetes Engine」>「Container Clusters」>「Create a container cluster」。

您可以在其他可用區中建立叢集，但建議將叢集保留在與容器登錄服務所用儲存空間值區相同的區域 (請參閱上一個步驟)。

您現在應該已擁有由 Google Kubernetes Engine 支援的完整 Kubernetes 叢集：

Kubernetes clusters

+ CREATE CLUSTER

+ DEPLOY

REFRESH

DELETE

SHOW

A Kubernetes cluster is a managed group of uniform VM instances for running Kubernetes. [Learn more](#)

Filter by label or name

☐ Name ^	Location	Cluster size	Total cores	Total memory	Notifications	Labels
☐ hello-cluster	us-central1-c	2	2 vCPUs	7.50 GB		Connect

現在，請將自己的容器化應用程式部署至 Kubernetes 叢集！從現在起，您將使用 `kubectl` 指令列 (已在 Cloud Shell 環境中設定)。本程式碼研究室的其餘部分需要 Kubernetes 用戶端和伺服器版本為 1.2 以上。`kubectl version` 會顯示指令的目前版本。

9. 將應用程式部署至 Kubernetes

Kubernetes 部署可使用您剛建立的容器映像檔，建立、管理及擴充應用程式的多個執行個體。讓我們使用 `kubectl create deployment` 指令，在 Kubernetes 中建立應用程式的部署作業：

```
$ kubectl create deployment hello-micronaut \
  --image=gcr.io/$GOOGLE_CLOUD_PROJECT/micronaut-jib:0.1
```

如要查看剛才建立的部署作業，只要執行下列指令即可：

```
$ kubectl get deployments
```

NAME	DESIRED	CURRENT	UP-TO-DATE	AVAILABLE	AGE
hello-micronaut	1	1	1	1	5m

如要查看部署作業建立的應用程式例項，請執行下列指令：

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
hello-micronaut-5647fb98c5-1h5h7	1/1	Running	0	5m

此時，您的容器應該已在 Kubernetes 的控管下執行，但您仍須讓外部存取該容器。

10. 允許外部流量

根據預設，Pod 只能透過叢集內的內部 IP 存取。如要從 Kubernetes 虛擬網路外部存取 `hello-micronaut` 容器，您必須將 Pod 公開為 Kubernetes 服務。

在 Cloud Shell 中，您可以結合 `kubectl expose` 指令和 `--type=LoadBalancer` 旗標，將 Pod 公開至網際網路。建立可從外部存取的 IP 時，必須使用這個旗標：

```
$ kubectl expose deployment hello-micronaut --type=LoadBalancer --port=8080
```

這個指令使用的旗標指定您將使用基礎架構提供的負載平衡器 (在本例中為 [Compute Engine 負載平衡器](#))。請注意，您公開的是部署作業，而不是 Pod。這會導致產生的服務在 Deployment 管理的所有 Pod 中，進行流量負載平衡 (在本例中只有 1 個 Pod，但您稍後會新增更多副本)。

Kubernetes 主節點會建立負載平衡器和相關的 Compute Engine 轉送規則、目標集區和防火牆規則，確保可從 Google Cloud Platform 外部完整存取服務。

如要找出服務的公開 IP 位址，只要要求 `kubectl` 列出所有叢集服務即可：

```
$ kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
hello-micronaut	LoadBalancer	10.39.243.251	aaa.bbb.ccc.ddd	8080:30354/TCP	1m
kubernetes	ClusterIP	10.39.240.1	<none>	443/TCP	31m

`EXTERNAL-IP` 可能需要幾分鐘才會顯示。如果找不到 `EXTERNAL-IP`，請稍候幾分鐘再試一次。

請注意，服務列出了 2 個 IP 位址，兩者都提供通訊埠 8080。一個是只能在雲端虛擬網路內看到的內部 IP，另一個是外部負載平衡 IP。在本範例中，外部 IP 位址為 `aaa.bbb.ccc.ddd`。

現在，只要在瀏覽器中前往這個網址：`http://<EXTERNAL_IP>:8080/hello`，就能連上服務。

11. 擴充服務

Kubernetes 的強大功能之一，就是能輕鬆擴充應用程式。假設您突然需要更多應用程式容量，只要告知複製控制器管理應用程式執行個體的新副本數量即可：

```
$ kubectl scale deployment hello-micronaut --replicas=3
deployment.extensions "hello-micronaut" scaled

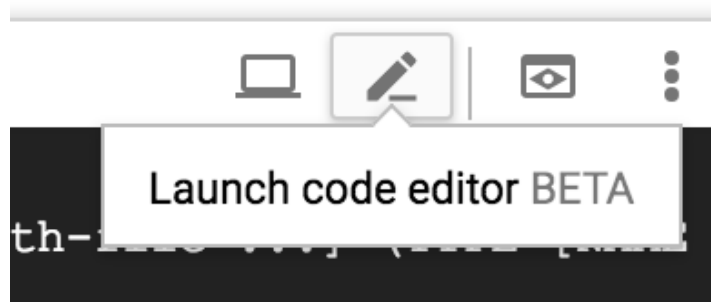
$ kubectl get deployment
NAME                DESIRED    CURRENT    UP-TO-DATE    AVAILABLE    AGE
hello-micronaut      3          3          3             3            16m
```

請注意，這裡採用的是**宣告式方法**，也就是宣告應隨時執行的執行個體數量，而不是啟動或停止新的執行個體。Kubernetes 調解迴圈只是確保實際情況符合您的要求，並視需要採取行動。

12. 推出服務升級

您部署到實際工作環境的應用程式，在某個時間點會需要修正錯誤或新增功能。Kubernetes 可協助您將新版本部署至正式環境，且不會影響使用者。

首先，請修改應用程式。從 Cloud Shell 開啟程式碼編輯器。



前往 `/jib/examples/micronaut/src/main/groovy/example/micronaut/HelloController.groovy`，然後更新回應的值：

```
@Controller("/hello")
class HelloController {
    @Get("/")
    String index() {
        "Hello Kubernetes World"
    }
}
```

在 `/jib/examples/micronaut/build.gradle` 中，我們會更新這一行，將映像檔版本從 0.1 升級至 0.2：

```
version '0.2'
```

然後重新建構及封裝應用程式，並套用最新變更：

```
$ ./gradlew jibDockerBuild
```

然後將映像檔加上標記並推送至容器映像檔登錄檔：

```
$ docker tag micronaut-jib:0.2 \
    gcr.io/$GOOGLE_CLOUD_PROJECT/micronaut-jib:0.2
$ docker push gcr.io/$GOOGLE_CLOUD_PROJECT/micronaut-jib:0.2
```

由於我們充分運用快取，建構及推送這個更新後的映像檔的速度應該會快上許多。

現在 Kubernetes 可以順利將複製控制器更新為新版應用程式。如要變更執行中容器的映像檔標籤，請編輯現有的 `hello-micronaut deployment`，並將映像檔從 `gcr.io/PROJECT_ID/micronaut-jib:0.1` 變更為

```
gcr.io/PROJECT_ID/micronaut-jib:0.2。
```

您可以使用 `kubectl set image` 指令，要求 Kubernetes 透過滾動更新，一次在整個叢集部署一個執行個體的新版應用程式：

```
$ kubectl set image deployment/hello-micronaut \
    micronaut-jib=gcr.io/$GOOGLE_CLOUD_PROJECT/micronaut-jib:0.2

deployment.apps "hello-micronaut" image updated
```

在此期間，服務使用者可能會遇到中斷情形。但這是因為沒有設定[健康狀態檢查](#)。這是稍微進階的設定，我們不會在本實驗室中新增健康狀態檢查。

再次檢查 http://EXTERNAL_IP:8080，確認傳回的是新回應。

步驟 13 · 約 2 分鐘

13. 復原

糟糕，您是否不小心上傳了新版應用程式？或許新版本含有錯誤，您需要快速還原。使用 Kubernetes 時，您可以輕鬆復原至先前的狀態。執行下列指令，復原應用程式：

```
$ kubectl rollout undo deployment/hello-micronaut
```

如果查看服務的輸出內容，會發現我們又回到最初的「Hello World」訊息。

14. 摘要

在本步驟中，您設定了以 Apache Groovy 為基礎的簡單 Micronaut Hello World 服務，並直接從 Cloud Shell 內執行該服務，然後使用 Jib 將其封裝為容器，並部署至 Google Kubernetes Engine。

步驟 15 · 約 0 分鐘

15. 恭喜！

您已學會如何建構新的 Apache Groovy / Micronaut 網頁型微服務，並部署至 Google Kubernetes Engine 上的 Kubernetes。

瞭解詳情

- Jib 說明文件和範例：<https://github.com/GoogleContainerTools/jib/>
- Micronaut 網站：<http://micronaut.io/>
- 在 Google Cloud Platform 上使用 Java：<https://cloud.google.com/java/>
- 如需 Java 範例，請參閱：<https://cloud.google.com/java/samples>
- 如需更完整詳細的 Kubernetes 教學課程，請參閱 bit.ly/k8s-lab，瞭解如何部署全端應用程式。

授權

這項內容採用的授權為 Creative Commons 姓名標示 2.0 通用授權。
